

Assetleum_CI/CD_계획서_정리본

1. 현재 인프라 요약

영역	구성
Front End	Vue + Vite → Amazon S3 → CloudFront
Back End	Spring Boot → Docker image → Amazon ECR → EC2 Docker
확장/복구	Launch Template + Auto Scaling Group + CloudWatch Alarm
트래픽	ALB + Target Group
운영 설정	AWS Parameter Store 에서 환경변수 조회 후 .env 생성
데이터 연결	RDS MariaDB, ElastiCache Redis, S3 파일 저장소
도메인	https://assetieum.com
CI/CD	GitHub Actions

Assetleum CI/CD 전체 흐름



Front End 자동 배포와 Back End ECR/Auto Scaling 배포 흐름

2. 프론트엔드 CI/CD 계획

정적 웹 배포: GitHub Actions → S3 → CloudFront

2.1 프론트엔드 배포 프로세스

1. 프론트엔드 코드 수정 후 GitHub 배포 브랜치에 Commit 및 Push
2. GitHub Actions를 통한 소스 코드 불러오기 및 빌드 환경 구성
3. 프로젝트 의존성 설치 및 프론트엔드 애플리케이션 빌드
4. 빌드 과정에서 운영 API 및 실시간 알림 서버 주소 적용
5. 생성된 정적 파일을 프론트엔드 배포용 S3 버킷에 업로드
6. CloudFront 캐시 무효화를 통한 최신 파일 반영
7. 운영 프론트엔드 접속 및 주요 화면 표시 확인
8. 백엔드 API 및 실시간 알림 연결 상태 확인

2.2 배포 전략

항목	계획
배포 방식	S3 정적 파일 교체와 CloudFront 캐시 무효화를 이용한 자동 배포

항목	계획
무중단 기준	정적 자산을 먼저 업로드한 뒤 캐시를 무효화하여 최신 파일을 제공
삭제 동기화	aws s3 sync --delete 적용
버전 보관	GitHub Actions에서 빌드한 Docker 이미지를 ECR에 github.sha와 latest 태그로 push하여 관리
롤백	이전 빌드 재업로드 절차: GitHub Actions의 Backend Rollback workflow를 수동 실행하여 이전 ECR 이미지 태그를 latest로 재태깅한 뒤 ASG Instance Refresh를 수행한다. 담당자: 배포 권한을 가진 백엔드/인프라 담당자
배포 로그	GitHub Actions 실행 로그와 EC2의 Docker 실행 로그를 통해 배포 결과를 확인하며, 배포 실패 시 컨테이너 상태와 최근 Docker 로그(최대 200줄)를 출력하여 원인 분석이 가능하도록 구성

2.3 보안 및 환경변수

- AWS 인증 방식: Access Key (GitHub Secret)
- GitHub Secrets/Variables 에는 실제 값만 저장하고 워크플로 파일에 민감 정보를 직접 작성하지 않습니다.
- VITE_API_BASE_URL: <https://api.assetieum.com/api/v1>
- VITE_SSE_BASE_URL: <https://api.assetieum.com/api/v1>
- S3 와 CloudFront 작업에 필요한 최소 IAM 권한 범위: AmazonS3FullAccess, AmazonEC2ContainerRegistryReadOnly, AmazonSSMManagedInstanceCore

2.4 프론트엔드 GitHub Actions 템플릿

아래 값은 실제 리소스 정보로 교체해야 실행할 수 있습니다.

```
name: Frontend Deploy

on:
  push:
    branches:
      - main
    workflow_dispatch:

concurrency:
  group: frontend-deploy
  cancel-in-progress: true

env:
  AWS_REGION: ${vars.AWS_REGION || 'ap-northeast-2'}
  NODE_VERSION: ${vars.NODE_VERSION || '24'}
  VITE_API_BASE_URL: ${vars.VITE_API_BASE_URL || '/api/v1'}
  VITE_SSE_BASE_URL: ${vars.VITE_SSE_BASE_URL || '/api/v1'}
  VITE_USE MOCKS: 'false'

jobs:
  test:
    name: Test
    runs-on: ubuntu-latest

    permissions:
      contents: read

    steps:
      - name: Checkout
        uses: actions/checkout@v4
```

```

- name: Set up Node.js
  uses: actions/setup-node@v4
  with:
    node-version: ${ env.NODE_VERSION }
    cache: npm

- name: Install dependencies
  run: npm ci

- name: Run lint
  run: npm run lint

- name: Build
  run: npm run build

deploy-s3-cloudfront:
  name: Deploy to S3 and CloudFront
  runs-on: ubuntu-latest
  needs: test

  permissions:
    contents: read

  env:
    S3_BUCKET: ${ vars.S3_BUCKET }
    CLOUDFRONT_DISTRIBUTION_ID: ${ vars.CLOUDFRONT_DISTRIBUTION_ID }

  steps:
    - name: Checkout
      uses: actions/checkout@v4

    - name: Validate deploy configuration
      run: |
        set -eu
        if [ -z "$S3_BUCKET" ]; then
          echo "Missing GitHub variable: S3_BUCKET"
          exit 1
        fi
        if [ -z "$CLOUDFRONT_DISTRIBUTION_ID" ]; then
          echo "Missing GitHub variable: CLOUDFRONT_DISTRIBUTION_ID"
          exit 1
        fi

    - name: Set up Node.js
      uses: actions/setup-node@v4
      with:
        node-version: ${ env.NODE_VERSION }
        cache: npm

    - name: Install dependencies
      run: npm ci

    - name: Build
      run: npm run build

    - name: Configure AWS credentials
      uses: aws-actions/configure-aws-credentials@v4
      with:
        aws-access-key-id: ${ secrets.AWS_ACCESS_KEY_ID }
        aws-secret-access-key: ${ secrets.AWS_SECRET_ACCESS_KEY }
        aws-region: ${ env.AWS_REGION }

    - name: Upload hashed assets
      run: |
        if [ -d dist/assets ]; then
          aws s3 sync dist/assets "s3://$S3_BUCKET/assets" \
            --delete \
            --cache-control "public,max-age=31536000,immutable"
        fi

    - name: Upload entry and public files

```

```

run: |
  aws s3 sync dist "s3://$S3_BUCKET" \
    --delete \
    --exclude "assets/*" \
    --cache-control "no-cache,no-store,must-revalidate"
- name: Invalidate CloudFront cache
  run: |
    aws cloudfront create-invalidation \
      --distribution-id "$CLOUDFRONT_DISTRIBUTION_ID" \
      --paths "/*"

```

3. 백엔드 CI/CD 계획

Docker 이미지 배포: GitHub Actions → ECR → EC2/Auto Scaling

3.1 현재 상태와 목표 상태

구분	현재 확인 내용	계획서 반영
빌드/테스트	GitHub Actions, 테스트용 H2, Redis service	자동화 유지
이미지 배포	Docker build 후 ECR(assetieum-be) push	자동화 유지
EC2 실행	ECR 이미지를 pull 하여 docker-compose 로 실행	자동화 유지
Auto Scaling 반영	ASG Instance Refresh 전환 준비	현재 방식

3.2 백엔드 배포 프로세스

1. 백엔드 코드 수정 후 GitHub 배포 브랜치에 Commit 및 Push
2. GitHub Actions를 통한 코드 빌드 및 테스트
3. 테스트 완료 후 백엔드 Docker 이미지 생성
4. 생성된 Docker 이미지를 AWS ECR의 `assetieum-be` 저장소에 업로드
5. Auto Scaling Group의 인스턴스 교체 작업 시작
6. 새로운 EC2 인스턴스 생성 및 운영 환경변수 설정
7. ECR에서 최신 백엔드 이미지 다운로드
8. Docker Compose를 통한 백엔드 애플리케이션 실행
9. ALB Health Check를 통한 신규 서버 정상 동작 확인
10. 정상 확인 후 기존 EC2 인스턴스 순차 교체
11. 배포 완료 후 API, Redis, RDS, S3 파일 업로드 기능 확인

3.3 Back End 배포 전략 및 운영 기준

3.4 배포 전략

항목	계획
배포 방식	ECR 이미지 기반 EC2 컨테이너 배포
전환 전략	ASG Instance Refresh 기반 순차 교체 - (MinHealthyPercentage = 50 , InstanceWarmup = 180)
트래픽 전환	새 인스턴스가 ALB Health Check 를 통과한 뒤 Target Group 에서 트래픽 수신

항목	계획
환경변수	Parameter Store → User Data/배포 스크립트 → .env 생성
자동 복구	ASG Self-Healing 으로 비정상 EC2 대체
스케일링	CloudWatch CPU Alarm 기반 Scale Out/Scale In Target CPU: 60% Instance Warmup: 300초 Min/Desired/Max: 1 / 1 / 3

3.5 롤백 계획

단계	필요 내용
1. 이전 버전 식별	ECR Images에서 롤백 대상 이미지 태그 확인
2. 실행 이미지 변경	GitHub Actions Rollback Workflow에서 <code>image_tag</code> 에 이전 SHA 입력
3. 재배포	Workflow 실행 후 ECR <code>latest</code> 재태깅 및 ASG Instance Refresh 수행
4. 검증	Health Check, 로그인, 핵심 API, 파일 업로드, Redis 연결 정상 여부 확인
5. 실패 대응	Workflow 실패 시 로그 확인, 필요 시 기존 정상 SHA로 재실행

3.6 보안 관리

- GitHub Actions 배포 인증 정보의 GitHub Secrets 저장 및 외부 노출 방지
- 배포 작업에 필요한 AWS 권한만 최소 범위로 부여
- EC2 인스턴스에 필요한 AWS 서비스 접근 권한만 IAM Role로 부여
- 운영 환경변수 및 민감 정보의 Parameter Store 관리
- `.env`, Access Key, 데이터베이스 비밀번호 등 민감 정보의 GitHub 저장소 업로드 금지
- 운영 환경설정 및 인증 정보의 Docker 이미지와 AMI 포함 금지
- S3, ECR, Parameter Store 등 AWS 리소스에 대한 접근 권한 제한
- ECR Docker 이미지 취약점 검사 및 오래된 이미지 정리
- 배포 및 서버 접근 기록 확인을 위한 로그 관리
- Access Key와 비밀번호 등 인증 정보의 주기적인 점검 및 변경

3.7 Back End GitHub Actions 템플릿 - Build & ECR Push

```
name: Backend Deploy

on:
  push:
    branches:
      - main
    workflow_dispatch:

concurrency:
  group: backend-deploy
  cancel-in-progress: true

env:
  AWS_REGION: ${ vars.AWS_REGION || 'ap-northeast-2' }
  ECR_REPOSITORY: ${ vars.ECR_REPOSITORY || 'assetium-be' }
```

```

jobs:
  test:
    name: Test
    runs-on: ubuntu-latest

    services:
      redis:
        image: redis:7-alpine
        ports:
          - 6379:6379
        options: >-
          --health-cmd "redis-cli ping"
          --health-interval 10s
          --health-timeout 5s
          --health-retries 5
    permissions:
      contents: read

    steps:
      - name: Checkout
        uses: actions/checkout@v4

      - name: Set up JDK 21
        uses: actions/setup-java@v4
        with:
          distribution: temurin
          java-version: '21'
          cache: gradle

      - name: Grant execute permission for Gradle
        run: chmod +x ./gradlew

      - name: Run tests
        run: ./gradlew clean test --no-daemon

  build-and-push:
    name: Build and Push Image
    runs-on: ubuntu-latest
    needs: test

    permissions:
      contents: read

    outputs:
      image-uri: ${ steps.image.outputs.image-uri }
      image-tag: ${ github.sha }

    steps:
      - name: Checkout
        uses: actions/checkout@v4

      - name: Configure AWS credentials
        uses: aws-actions/configure-aws-credentials@v4
        with:
          aws-access-key-id: ${ secrets.AWS_ACCESS_KEY_ID }
          aws-secret-access-key: ${ secrets.AWS_SECRET_ACCESS_KEY }
          aws-region: ${ env.AWS_REGION }

      - name: Login to Amazon ECR
        id: login-ecr
        uses: aws-actions/amazon-ecr-login@v2

      - name: Set image names
        id: image
        run: |
          IMAGE_REPOSITORY="${ steps.login-ecr.outputs.registry }/${ env.ECR_REPOSITORY }"
          echo "image-repository=$IMAGE_REPOSITORY" >> "$GITHUB_OUTPUT"
          echo "image-uri=$IMAGE_REPOSITORY:${ github.sha }" >> "$GITHUB_OUTPUT"

      - name: Set up Docker Buildx
        uses: docker/setup-buildx-action@v3

```

```

- name: Build and push Docker image
  uses: docker/build-push-action@v6
  with:
    context: .
    push: true
    tags: |
      ${ steps.image.outputs.image-repository }:${ github.sha }
      ${ steps.image.outputs.image-repository }:latest
    labels: |
      org.opencontainers.image.source=${ github.server_url }/${ github.repository }
      org.opencontainers.image.revision=${ github.sha }
    cache-from: type=gha
    cache-to: type=gha,mode=max

refresh-asg:
  name: Refresh Auto Scaling Group
  runs-on: ubuntu-latest
  needs: build-and-push

permissions:
  contents: read

steps:
- name: Configure AWS credentials
  uses: aws-actions/configure-aws-credentials@v4
  with:
    aws-access-key-id: ${ secrets.AWS_ACCESS_KEY_ID }
    aws-secret-access-key: ${ secrets.AWS_SECRET_ACCESS_KEY }
    aws-region: ${ env.AWS_REGION }

- name: Start ASG instance refresh
  run: |
    aws autoscaling start-instance-refresh \
      --auto-scaling-group-name "${ secrets.ASG_NAME }" \
      --preferences MinHealthyPercentage=50,InstanceWarmup=180

```

4. Back End 서버 반영 단계

4.1 ASG Instance Refresh

Launch Template/User Data 가 새 이미지 태그를 읽어 실행할 수 있어야 합니다. 이미지 참조 갱신이 빠지면 새 EC2 도 이전 이미지를 실행할 수 있습니다.

```

- name: Start ASG instance refresh
  run: |
    aws autoscaling start-instance-refresh \
      --auto-scaling-group-name "${ secrets.ASG_NAME }" \
      --preferences MinHealthyPercentage=50,InstanceWarmup=180

```

4.3 EC2 부팅/배포 스크립트 필수 항목

순서	EC2에서 수행할 작업
1	AWS Systems Manager Parameter Store 에서 운영 환경변수 조회 Parameter 경로: <code>/assetieum/prod/env</code>
2	조회한 환경변수를 .env 파일로 생성 생성 경로: <code>/home/ubuntu/<PROJECT_DIRECTORY>/ .env</code>
3	Amazon ECR 로그인 후 Docker 이미지 다운로드(Pull) ECR Registry: <code><AWS_ACCOUNT_ID>.dkr.ecr.<REGION>.amazonaws.com</code>

순서	EC2에서 수행할 작업
	Repository: <code>assetieum-be</code> Image Tag: <code>latest</code>
4	Docker Compose를 이용하여 애플리케이션 실행 docker-compose.yml 경로: <code>/home/ubuntu/<PROJECT_DIRECTORY>/docker-compose.yml</code> 실행 명령: <code>docker compose pull</code> → <code>docker compose up -d</code>
5	애플리케이션 Health Check 확인서비스 포트: <code>8080</code> Health Check API: <code>GET /api/v1/health</code> (<i>HealthCheckController 직접 구현</i>) 확인 명령: <code>curl http://localhost:8080/api/v1/health</code>
6	CloudWatch: CPUUtilization 모니터링, Target Tracking 정책, Scale Out/Scale In 알람 확인 애플리케이션 로그: <code>docker compose logs -f</code> 또는 <code>docker logs <컨테이너명></code>